Clinic Management System (CMS) — SaaS Platform

Portfolio Project Documentation
Stack: MERN (MongoDB, Express.js, React, Node.js)
Type: SaaS Multi-Tenant Application
Version: 1.0.0

---

1. Project Overview

The **Clinic Management System** is a full-stack SaaS platform designed to streamline operations for medical clinics, private practices, and healthcare providers. It enables clinics to manage patient records, appointments, prescriptions, billing, and staff roles from a single dashboard.

The platform is built as a **multi-tenant SaaS** architecture, meaning multiple clinics can register, operate independently, and manage their own data in isolated environments.

Key Highlights

- Multi-tenant SaaS architecture with role-based access control (RBAC)
- Real-time appointment scheduling with conflict detection
- Digital prescription generation and medical history tracking
- Automated billing, invoicing, and payment status tracking
- Responsive React dashboard with dark/light mode support
- RESTful API with JWT authentication and refresh token rotation

---

2. Tech Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	UI framework with fast HMR
State Management	Redux Toolkit + RTK Query	Global state & server state caching
Styling	Tailwind CSS + shadcn/ui	Utility-first styling & component library
Routing	React Router v6	SPA navigation with protected routes
Backend	Node.js + Express.js	REST API server
Authentication	JWT (Access + Refresh tokens) + bcrypt	Secure auth with token rotation
Database	MongoDB + Mongoose	NoSQL document store with schema validation
File Storage	Cloudinary / AWS S3	Medical document & avatar uploads
Real-time	Socket.io	Live notifications & appointment updates
Email	Nodemailer + SendGrid	Appointment reminders & password reset

Generated by Kimi.ai

Table 1 – continued

Layer	Technology	Purpose
Validation	Zod (shared)	Runtime schema validation across stack
Testing	Jest + React Testing Library + Supertest	Unit & integration tests
Deployment	Docker + Render / AWS EC2	Containerized deployment

---

3. Core Features

3.1 Authentication & Authorization

- Secure registration/login with JWT (access token: 15min, refresh token: 7 days)
- Role-based access control: **Super Admin**, **Clinic Admin**, **Doctor**, **Receptionist**, **Patient**
- Password reset via secure email token
- Session management with "Log out all devices" functionality

3.2 Clinic Management (Multi-Tenant)

- Clinic onboarding with subdomain or custom slug (`clinic-name.app.com`)
- Clinic profile management (logo, address, contact, specialties)
- Subscription & billing tier management (Free, Pro, Enterprise)
- Staff member invitation and role assignment

3.3 Patient Management

- Digital patient registration with unique Patient ID
- Complete medical history (allergies, chronic conditions, surgeries)
- Document upload (lab reports, X-rays, prescriptions)
- Emergency contact & insurance information
- Patient portal for self-service appointment booking

3.4 Appointment System

- Drag-and-drop calendar view (day/week/month) for doctors
- Real-time slot availability with conflict prevention
- Appointment types: Walk-in, Scheduled, Telemedicine, Follow-up
- Automated SMS/email reminders (24h, 1h before)
- Check-in/check-out tracking with wait time analytics

3.5 Electronic Health Records (EHR)

- SOAP note creation and templating
- Digital prescription generation with drug database lookup
- Vital signs tracking with chart visualization
- Referral letter generation and sharing
- Audit log for all record modifications (HIPAA compliance)

3.6 Billing & Invoicing

- Service catalog with custom pricing per clinic
- Invoice generation with PDF export
- Payment tracking (Cash, Card, Insurance, Pending)
- Financial dashboard with revenue analytics
- Insurance claim tracking status

3.7 Reporting & Analytics

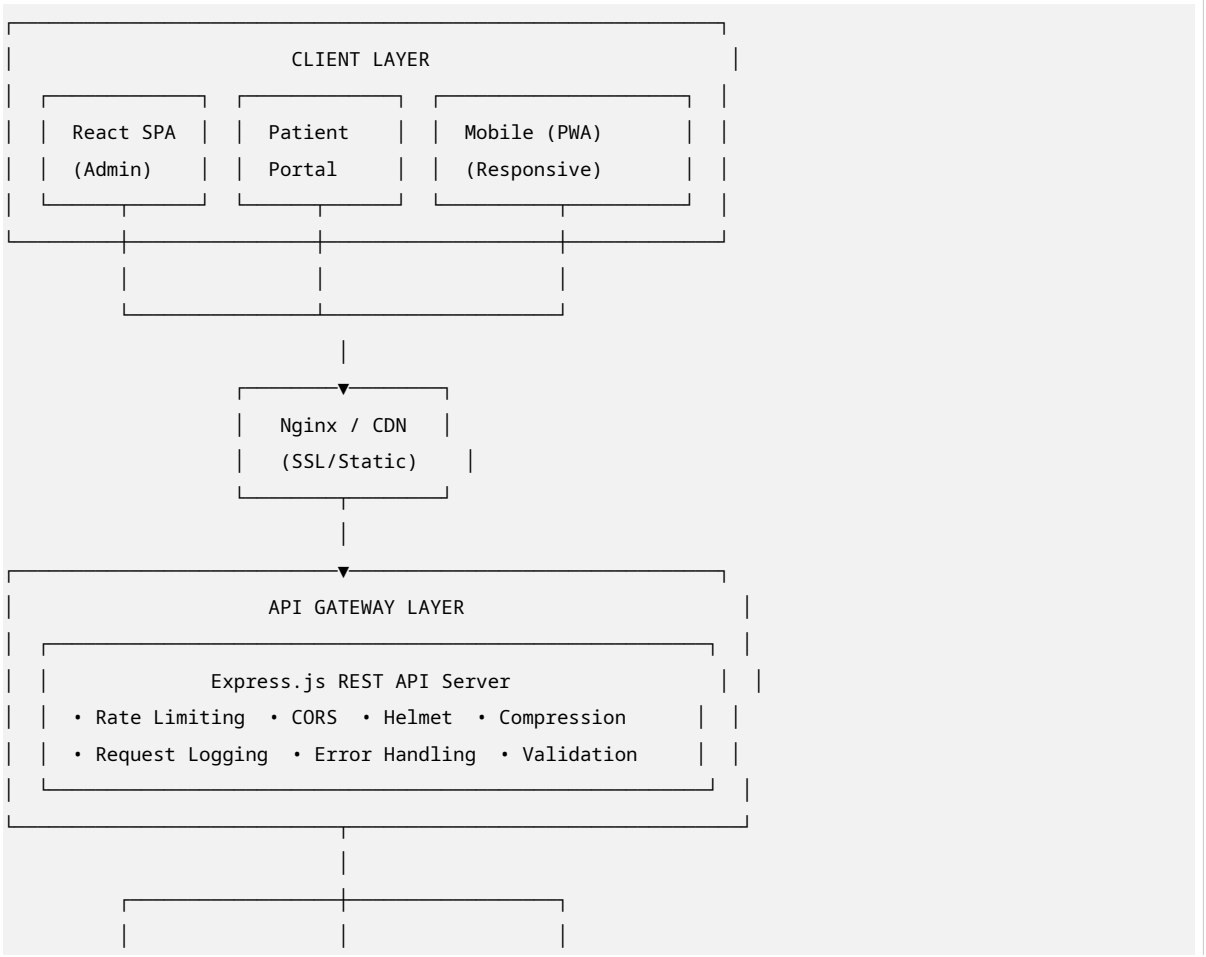
- Clinic performance dashboard (appointments, revenue, patient volume)
- Doctor workload and availability reports
- Patient demographics and visit frequency analytics
- Exportable reports (PDF, CSV)

3.8 Notifications & Communication

- In-app notification center
- Email notifications for appointments, prescriptions, billing
- SMS integration for urgent reminders
- Internal messaging between staff members

---

4. System Architecture



```
    ┌──────▼──────┐    ┌──────▼──────┐    ┌──────▼──────┐
    │  MongoDB    │    │  Redis      │    │ Cloudinary  │
    │  (Atlas)    │    │  (Sessions) │    │  (Files)    │
    │  • Tenant   │    │  • Cache    │    │  • Images   │
    │    Isolation │    │  • RateLimit│    │  • PDFs     │
    └─────────────┘    └─────────────┘    └─────────────┘
```

---

5. Database Schema Design

5.1 Core Collections

```
// Clinic (Tenant isolation root)
{
  _id: ObjectId,
  name: String,
  slug: String,              // Unique subdomain identifier
  email: String,
  phone: String,
  address: { street, city, state, zip, country },
  logo: String,              // Cloudinary URL
  subscription: {
    plan: Enum['free', 'pro', 'enterprise'],
    status: Enum['active', 'trial', 'expired'],
    expiresAt: Date
  },
  settings: {
    timezone: String,
    currency: String,
    appointmentSlots: Number // minutes per slot
  },
  createdAt: Date,
  updatedAt: Date
}

// User (Staff & Patients)
{
  _id: ObjectId,
  clinicId: ObjectId,        // Tenant reference
  email: String,
  password: String,          // bcrypt hashed
  role: Enum['super_admin', 'clinic_admin', 'doctor', 'receptionist', 'patient'],
  profile: {
    firstName: String,
    lastName: String,
    avatar: String,
    phone: String,
    dateOfBirth: Date,
    gender: Enum['male', 'female', 'other']
  },
  isActive: Boolean,
```

```
  lastLogin: Date,
  createdAt: Date
}

// Patient (Extended profile for patients)
{
  _id: ObjectId,
  userId: ObjectId,
  clinicId: ObjectId,
  patientId: String,         // Human-readable ID (e.g., P-2024-001)
  bloodGroup: String,
  allergies: [String],
  chronicConditions: [String],
  emergencyContact: { name, phone, relationship },
  insurance: { provider, policyNumber, expiryDate },
  medicalHistory: [{
    condition: String,
    diagnosedDate: Date,
    status: Enum['active', 'resolved']
  }],
  documents: [{ name, url, type, uploadedAt }]
}

// Doctor (Extended profile for doctors)
{
  _id: ObjectId,
  userId: ObjectId,
  clinicId: ObjectId,
  specialization: String,
  licenseNumber: String,
  experience: Number,        // Years
  education: [{ degree, institution, year }],
  schedule: {
    monday: [{ start: String, end: String }],
    tuesday: [...],
    // ... other days
  },
  consultationFee: Number,
  rating: Number,
  isAvailable: Boolean
}

// Appointment
{
  _id: ObjectId,
  clinicId: ObjectId,
  patientId: ObjectId,
  doctorId: ObjectId,
  appointmentNumber: String,  // Auto-generated
  type: Enum['walk_in', 'scheduled', 'telemedicine', 'follow_up'],
```

```
  status: Enum['pending', 'confirmed', 'checked_in', 'in_progress', 'completed', 'cancelled',
  ↪  'no_show'],
  scheduledDate: Date,
  duration: Number,          // Minutes
  reason: String,
  notes: String,
  checkedInAt: Date,
  completedAt: Date,
  createdBy: ObjectId,       // Receptionist or Patient
  createdAt: Date
}

// Medical Record (SOAP Notes)
{
  _id: ObjectId,
  clinicId: ObjectId,
  patientId: ObjectId,
  appointmentId: ObjectId,
  doctorId: ObjectId,
  subjective: String,         // Patient complaints
  objective: String,          // Examination findings
  assessment: String,         // Diagnosis
  plan: String,               // Treatment plan
  diagnosis: [{ code: String, name: String, type: Enum['primary', 'secondary'] }],
  vitals: {
    bloodPressure: String,
    heartRate: Number,
    temperature: Number,
    weight: Number,
    height: Number
  },
  attachments: [{ name, url }],
  createdAt: Date,
  updatedAt: Date
}

// Prescription
{
  _id: ObjectId,
  clinicId: ObjectId,
  patientId: ObjectId,
  doctorId: ObjectId,
  recordId: ObjectId,
  medications: [{
    name: String,
    dosage: String,
    frequency: String,
    duration: String,
    instructions: String
  }],
  status: Enum['active', 'completed'],
```

```
  issuedAt: Date,
  validUntil: Date
}

// Invoice
{
  _id: ObjectId,
  clinicId: ObjectId,
  patientId: ObjectId,
  appointmentId: ObjectId,
  invoiceNumber: String,
  items: [{
    description: String,
    quantity: Number,
    unitPrice: Number,
    total: Number
  }],
  subtotal: Number,
  tax: Number,
  discount: Number,
  total: Number,
  paid: Number,
  balance: Number,
  status: Enum['draft', 'sent', 'paid', 'partial', 'overdue', 'cancelled'],
  paymentMethod: Enum['cash', 'card', 'insurance', 'online'],
  dueDate: Date,
  paidAt: Date,
  createdAt: Date
}

// Audit Log (HIPAA Compliance)
{
  _id: ObjectId,
  clinicId: ObjectId,
  userId: ObjectId,
  action: String,             // e.g., 'VIEW_PATIENT', 'UPDATE_RECORD'
  resource: String,           // Collection name
  resourceId: ObjectId,
  details: Object,           // Change diff
  ipAddress: String,
  userAgent: String,
  timestamp: Date
}
```

---

6. API Endpoints

6.1 Authentication (`/api/v1/auth`)

Method	Endpoint	Description	Access
POST	`/register`	Register new clinic (Super Admin)	Public
POST	`/login`	Login with credentials	Public
POST	`/refresh`	Refresh access token	Public
POST	`/logout`	Logout and invalidate tokens	Authenticated
POST	`/forgot-password`	Request password reset	Public
POST	`/reset-password`	Reset password with token	Public
GET	`/me`	Get current user profile	Authenticated
PUT	`/me`	Update profile	Authenticated

6.2 Clinic Management (`/api/v1/clinics`)

Method	Endpoint	Description	Access
GET	`/`	List all clinics (Super Admin)	Super Admin
GET	`/:slug`	Get clinic by slug	Authenticated
PUT	`/:id`	Update clinic settings	Clinic Admin
GET	`/:id/stats`	Get clinic dashboard stats	Clinic Admin
POST	`/:id/staff`	Invite staff member	Clinic Admin
GET	`/:id/staff`	List all staff	Clinic Admin
PUT	`/:id/staff/:userId`	Update staff role	Clinic Admin
DELETE	`/:id/staff/:userId`	Remove staff member	Clinic Admin

6.3 Patients (`/api/v1/patients`)

Method	Endpoint	Description	Access
GET	`/`	List patients (paginated, searchable)	Staff
POST	`/`	Register new patient	Staff
GET	`/:id`	Get patient details	Staff
PUT	`/:id`	Update patient info	Staff
GET	`/:id/history`	Get medical history	Staff
GET	`/:id/documents`	Get patient documents	Staff
POST	`/:id/documents`	Upload document	Staff
DELETE	`/:id/documents/:docId`	Remove document	Clinic Admin

6.4 Appointments (`/api/v1/appointments`)

Method	Endpoint	Description	Access
GET	`/`	List appointments (filter by date/ doctor/status)	Staff
POST	`/`	Create appointment	Staff / Patient
GET	`/:id`	Get appointment details	Staff
PUT	`/:id`	Update appointment	Staff
PATCH	`/:id/status`	Update status (check-in, complete, cancel)	Staff
DELETE	`/:id`	Cancel appointment	Staff
GET	`/calendar`	Get calendar data for date range	Staff
GET	`/slots`	Get available slots for doctor & date	Public/Staff

6.5 Medical Records (`/api/v1/records`)

Method	Endpoint	Description	Access
GET	`/patient/:patientId`	List patient records	Staff
POST	`/`	Create medical record (SOAP)	Doctor
GET	`/:id`	Get record details	Staff
PUT	`/:id`	Update record	Doctor (own records)
POST	`/:id/prescription`	Add prescription to record	Doctor
GET	`/:id/prescription`	Get prescription	Staff

6.6 Billing (`/api/v1/billing`)

Method	Endpoint	Description	Access
GET	`/invoices`	List invoices	Staff
POST	`/invoices`	Create invoice	Staff
GET	`/invoices/:id`	Get invoice details	Staff
PUT	`/invoices/:id`	Update invoice	Staff
PATCH	`/invoices/:id/payment`	Record payment	Staff
GET	`/invoices/:id/pdf`	Download invoice PDF	Staff
GET	`/reports/revenue`	Revenue analytics	Clinic Admin

6.7 Notifications (`/api/v1/notifications`)

Method	Endpoint	Description	Access
GET	`/`	Get user notifications	Authenticated
PATCH	`/:id/read`	Mark as read	Authenticated
PATCH	`/read-all`	Mark all as read	Authenticated
DELETE	`/:id`	Delete notification	Authenticated

7. Frontend Architecture

7.1 Folder Structure

```
client/
├── public/
│   └── assets/
├── src/
│   ├── api/                # RTK Query API slices
│   │   ├── authApi.js
│   │   ├── clinicApi.js
│   │   ├── patientApi.js
│   │   ├── appointmentApi.js
│   │   ├── recordApi.js
│   │   └── billingApi.js
│   ├── components/          # Reusable UI components
│   │   ├── ui/             # shadcn/ui components
│   │   ├── layout/         # Sidebar, Navbar, Footer
│   │   ├── forms/          # Form components
│   │   ├── tables/         # Data tables with sorting/filtering
│   │   └── charts/         # Recharts wrappers
│   ├── features/           # Redux slices
│   │   ├── authSlice.js
│   │   ├── clinicSlice.js
│   │   └── uiSlice.js      # Theme, sidebar state
│   ├── hooks/              # Custom React hooks
│   │   ├── useAuth.js
│   │   ├── useClinic.js
│   │   └── useNotification.js
│   ├── pages/              # Route-level components
│   │   ├── auth/
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   └── ForgotPassword.jsx
│   │   ├── dashboard/
│   │   │   └── Dashboard.jsx
│   │   ├── patients/
│   │   │   ├── PatientList.jsx
│   │   │   ├── PatientDetail.jsx
│   │   │   └── PatientForm.jsx
│   │   ├── appointments/
│   │   │   ├── CalendarView.jsx
│   │   │   ├── AppointmentList.jsx
│   │   │   └── BookAppointment.jsx
│   │   ├── records/
│   │   │   ├── MedicalRecordList.jsx
│   │   │   ├── SOAPForm.jsx
│   │   │   └── PrescriptionForm.jsx
│   │   ├── billing/
│   │   │   ├── InvoiceList.jsx
│   │   │   ├── InvoiceDetail.jsx
│   │   │   └── PaymentForm.jsx
```

```
│   │   ├── staff/
│   │   │   └── StaffManagement.jsx
│   │   └── settings/
│   │       └── ClinicSettings.jsx
│   ├── routes/             # Route definitions & guards
│   │   ├── AppRoutes.jsx
│   │   └── ProtectedRoute.jsx
│   ├── utils/              # Helpers & constants
│   │   ├── constants.js
│   │   ├── formatters.js   # Date, currency formatters
│   │   └── validators.js   # Zod schemas
│   ├── styles/
│   │   └── globals.css
│   ├── App.jsx
│   └── main.jsx
├── .env
├── tailwind.config.js
├── vite.config.js
└── package.json
```

7.2 Key UI/UX Decisions

- **Responsive Design:** Mobile-first approach; sidebar collapses to drawer on mobile
- **Theme:** Light/dark mode toggle with system preference detection
- **Data Tables:** Sortable, filterable, paginated tables with export to CSV
- **Calendar:** FullCalendar.js integration for appointment scheduling
- **Charts:** Recharts for analytics (revenue trends, patient volume, etc.)
- **Forms:** React Hook Form + Zod for validation with real-time error feedback
- **Loading States:** Skeleton loaders for all data-fetching components
- **Toast Notifications:** Sonner for action confirmations and errors

---

8. Backend Architecture

8.1 Folder Structure

```
server/
├── config/
│   ├── database.js         # MongoDB connection with tenant isolation
│   ├── redis.js            # Redis client setup
│   └── cloudinary.js       # File upload config
├── controllers/            # Request handlers
│   ├── authController.js
│   ├── clinicController.js
│   ├── patientController.js
│   ├── appointmentController.js
│   ├── recordController.js
│   ├── billingController.js
│   └── notificationController.js
├── middleware/
```

```
│   ├── auth.js             # JWT verification & role checking
│   ├── tenant.js           # Tenant context extraction from subdomain/header
│   ├── errorHandler.js     # Centralized error handling
│   ├── validate.js         # Zod request validation
│   ├── upload.js           # Multer + Cloudinary upload middleware
│   └── rateLimiter.js      # API rate limiting per tenant
├── models/                 # Mongoose schemas
│   ├── Clinic.js
│   ├── User.js
│   ├── Patient.js
│   ├── Doctor.js
│   ├── Appointment.js
│   ├── MedicalRecord.js
│   ├── Prescription.js
│   ├── Invoice.js
│   └── AuditLog.js
├── routes/                 # Express route definitions
│   ├── authRoutes.js
│   ├── clinicRoutes.js
│   ├── patientRoutes.js
│   ├── appointmentRoutes.js
│   ├── recordRoutes.js
│   ├── billingRoutes.js
│   └── notificationRoutes.js
├── services/               # Business logic layer
│   ├── authService.js
│   ├── appointmentService.js
│   ├── billingService.js
│   ├── notificationService.js
│   └── pdfService.js       # PDF generation for invoices/prescriptions
├── utils/
│   ├── generateToken.js
│   ├── generateId.js       # Patient ID, Invoice number generators
│   ├── emailTemplates.js
│   └── logger.js           # Winston logger
├── jobs/                   # Background jobs (BullMQ)
│   ├── appointmentReminder.js
│   └── subscriptionRenewal.js
├── tests/
│   ├── unit/
│   └── integration/
├── app.js                  # Express app setup
├── server.js               # Entry point
├── .env
└── package.json
```

8.2 Security Measures

- **Helmet.js:** HTTP security headers
- **CORS:** Whitelist-based cross-origin policy
- **Rate Limiting:** 100 requests/minute per IP, stricter for auth endpoints

- **Input Sanitization:** MongoDB injection prevention via Mongoose + validation
- **Password Policy:** Min 8 chars, 1 uppercase, 1 number, 1 special char
- **File Upload:** Restricted to images/PDFs, max 5MB, scanned by Cloudinary
- **Audit Logging:** Every CRUD operation on patient data is logged with user IP & timestamp
- **Data Isolation:** All queries automatically filtered by `clinicId` (tenant middleware)

---

9. Environment Variables

Client (`.env`)

```
VITE_API_BASE_URL=https://api.clinicms.com/api/v1
VITE_SOCKET_URL=wss://api.clinicms.com
VITE_APP_NAME=ClinicMS
VITE_MAX_FILE_SIZE=5242880
```

Server (`.env`)

```
NODE_ENV=production
PORT=5000

# Database
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/clinicms?retryWrites=true&w=majority

# JWT
JWT_ACCESS_SECRET=your-super-secret-access-key
JWT_REFRESH_SECRET=your-super-secret-refresh-key
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d

# Redis
REDIS_URL=redis://localhost:6379

# Cloudinary
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret

# Email
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASS=your-sendgrid-api-key
FROM_EMAIL=noreply@clinicms.com

# Frontend URL
CLIENT_URL=https://app.clinicms.com
```

---

10. Development Setup

Prerequisites

- Node.js v18+
- MongoDB (local or Atlas)
- Redis (local or cloud)
- Git

Installation

```
# Clone repository
git clone https://github.com/yourusername/clinic-management-system.git
cd clinic-management-system

# Install server dependencies
cd server
npm install

# Install client dependencies
cd ../client
npm install

# Configure environment variables
cp .env.example .env
# Edit .env with your credentials

# Start development servers
# Terminal 1 (Backend)
cd server
npm run dev

# Terminal 2 (Frontend)
cd client
npm run dev
```

Available Scripts

Server:

- `npm run dev` — Start with nodemon
- `npm start` — Start production server
- `npm test` — Run Jest tests
- `npm run test:watch` — Run tests in watch mode

Client:

- `npm run dev` — Start Vite dev server
- `npm run build` — Production build
- `npm run preview` — Preview production build

- `npm test` — Run React Testing Library tests

---

11. Deployment

Docker Setup

```dockerfile
# Dockerfile (Server)
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

```dockerfile
# Dockerfile (Client - Nginx)
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

docker-compose.yml

```yaml
version: '3.8'
services:
  app:
    build: ./server
    ports:
      - "5000:5000"
    env_file: ./server/.env
    depends_on:
      - redis

  client:
    build: ./client
    ports:
      - "80:80"

  redis:
    image: redis:alpine
    ports:
      - "6379:6379"
```

Production Deployment Checklist

- Use MongoDB Atlas with VPC peering
- Enable SSL/TLS (Let's Encrypt or Cloudflare)
- Configure PM2 for Node.js process management
- Set up log rotation (Winston + external service)
- Configure backup strategy (MongoDB automated backups)
- Set up monitoring (New Relic / Datadog / UptimeRobot)
- Enable CDN for static assets (Cloudflare / AWS CloudFront)
- Configure CI/CD pipeline (GitHub Actions → Render/AWS)

---

12. Testing Strategy

Unit Tests

- **Controllers:** Test request/response handling with mocked services
- **Services:** Test business logic with mocked database calls
- **Utils:** Test helper functions and formatters

Integration Tests

- **API Endpoints:** Test full request lifecycle (auth → action → response)
- **Database:** Test schema validation and relationships
- **File Upload:** Test Cloudinary integration with mock uploads

Frontend Tests

- **Components:** Test rendering and user interactions
- **Pages:** Test routing and data fetching
- **Hooks:** Test custom hook logic

Test Coverage Goals

- Backend: >80% coverage
- Frontend: >70% coverage

---

13. Future Roadmap

Phase 2 (Short Term)

- Telemedicine integration (WebRTC video calls)
- SMS gateway integration (Twilio)
- Multi-language support (i18n)
- Mobile app (React Native)
- Inventory management (pharmacy stock)

Phase 3 (Long Term)

- AI-powered diagnosis suggestions

Generated by Kimi.ai

- Patient portal mobile app
- Integration with lab systems (HL7/FHIR)
- Insurance API integrations
- White-label solution for resellers

---

14. License & Credits

License: MIT
Author: [Your Name]
Portfolio: [Your Portfolio URL]
GitHub: [Your GitHub URL]

Built with care for the healthcare community.

This documentation serves as the complete technical specification and developer guide for the Clinic Management System SaaS platform. For questions or contributions, please open an issue on the GitHub repository.

Generated by Kimi.ai